

G.O.C Academy Hub

Development handover — what is finished, and what is left to build

This document records the state of the enhancement work on the existing G.O.C Academy Hub web application as at 31 August 2026. Sections 1 and 2 cover what has been built and verified. Section 3 is the part that matters for picking the work back up: every outstanding item, what has already been prepared for it, and what still has to be written. Sections 4 to 6 cover how to run and verify the project, the rules the work must not break, and the few decisions that are still yours to make.

At a glance	
Built and test-covered	Priorities 3, 4, 5, 6, 7, 8 and 9, plus both trailing instructions (UTME-only registration, and coins removed in favour of XP, streaks and levels)
Built, tests still owed	Priorities 10, 11 and 12 — the Scholar League
Not started	Priorities 1 (animations), 2 (Update & News navigation), 14 (device pass)
Running throughout	Priority 13 (security and data integrity), 15 (debugging), 16 (final verification)
Automated tests	881 assertions across 7 suites, all passing
Files changed so far	index.html, css/styles.css, js/app.js, js/api.js, js/goc-core.js, server/server.js, test/ (7 suites + runner + minidom)
Dependencies added	None. No build step, no bundler, no CDN links.
Logo	Untouched, as required.

1. Where the project stands

The application is still the same G.O.C Academy Hub: a single-page app made of index.html for structure, css/styles.css for appearance, js/app.js for behaviour, js/api.js as the only data-aware file, and js/goc-core.js as the shared domain core that both the browser and the Node server load. Nothing about that shape has changed, and nothing has been rebuilt.

Every feature added so far exists in both data modes. js/api.js probes for a server on start-up and picks one of two drivers: the demo driver, which runs entirely in the page so index.html still works when opened straight from disk, and the HTTP driver, which talks to server/server.js. Because both drivers share js/goc-core.js, a rule such as how a theory paper is totalled, or how a league is ordered, is written once and enforced identically in both.

Authorisation is decided by the data layer, never by the page. The server settles every request against one of five levels — signed in, student, staff, staff who have also passed the access passcode, and Founder — before any handler runs. The demo driver mirrors the same gates. This is why hiding a button is never the control; it is only the courtesy.

2. What is finished and verified

Each item below has been built, driven through its real interface by an automated test, and left passing.

Priority	What was delivered	Proven by
5 — UTME only, and the subject rule	Sign-up accepts UTME alone and refuses every other exam with an on-screen explanation. Use of English is compulsory and exactly three of the four sciences are required. The rule lives in <code>js/goc-core.js</code> , so the form and the API validate with the same code.	<code>test-rules.js</code> , <code>test-p6.js</code> , <code>test-server.js</code>
6 — the dashboard follows the student's subjects	The stored combination is the single source of truth. Video sessions, Study, Practice, Tests, materials, subject progress and subject performance all read from it rather than from a hard-coded list.	<code>test-p6.js</code>
3 — the Web Test page	A separate Web Test page with weekly and monthly papers and three separated sessions: theory (stored for a human to mark), objective (marked automatically with the full breakdown), and a JAMB-oriented CBT mode with numbering, navigation, answered indicators, remaining time and submit.	<code>test-p3.js</code> , <code>test-server.js</code>
4 — the console's Web Test management	The existing question bank was extended rather than duplicated: test type, period, subject, session, topic, question type, difficulty, explanation and an active switch, with theory questions carrying a reference answer and a maximum mark. Results are filed against the Scholar ID, objective papers are marked automatically, and theory papers are marked by hand.	<code>test-p4.js</code> , <code>test-server.js</code>
7 — account deactivation	Management can close and reopen a student account from the console. Closing withdraws access, drops the live session at once and flags the card, while the Scholar ID, results, marks, performance history and XP all stay on file and return intact when the account is reopened.	<code>test-p78.js</code> , <code>test-server.js</code>
8 — founder-only credentials	The management login password and the firewall passcode can only be changed by the Founder, and the server decides that. Passwords are hashed, never stored in plain text, never returned by an API and never rendered; the current passcode appears only as a row of dots.	<code>test-p78.js</code> , <code>test-server.js</code>
9 — the student session threshold	Management sets the session length and the warning lead time from the console. A student is warned before expiry with a live countdown reading "Automatic logout in 04:59", and at zero the session ends, the login screen explains why, and every protected screen is refused. The server owns the expiry; the page only displays it.	<code>test-p9.js</code> , <code>test-rules.js</code> , <code>test-server.js</code>
Trailing request — no in-app coins	Coins were removed throughout. Earning is now XP, streaks and levels only.	<code>test-p6.js</code> , <code>test-p3.js</code>

The session edge cases the specification names, and how they now behave

Case	Behaviour, as built and tested
Page refresh	The token lives in a variable and never in storage, so a refresh signs the student out by design rather than leaving a session recoverable.
Multiple tabs	Every tab asks the server, and the server holds one absolute expiry per token, so all tabs warn and end together. No cross-tab messaging is needed.
Network interruption	A failed poll never logs anyone out. The count carries on from the last known expiry and the failure is recorded; only the server's answer, or that countdown genuinely reaching zero, ends the session.
Protected screen after expiry	Refused twice over: by the guard inside <code>go()</code> , and by the server, which drops an expired token before any handler runs.
Work in progress	A submitted paper is already stored against the Scholar ID and is unaffected. An unsubmitted paper is reported in plain words as not submitted, rather than disappearing silently.

Case	Behaviour, as built and tested
A setting changed mid-session	A session already running is neither cut short nor prolonged — the new length applies at the next login — but a changed warning lead time reaches a student who is already signed in.
Management's own session	Staff keep a fixed eight-hour working session, so shortening a student session never shortens the console's.

Two faults were found by this testing rather than by reading the code, and both are worth knowing about. In `js/goc-core.js` the function that applies theory marks treated an unmarked answer as a zero, so saving a single mark silently awarded nothing to the rest of the paper, totalled it and declared it finished — exactly the automatic theory score the specification forbids. A half-marked paper now keeps no score at all until every written answer has been given one by hand. Separately, the demo driver was giving management the student session length, contradicting the console's own promise that staff sessions are unaffected; staff now hold the same eight hours the server gives them.

3. What remains to be finished

Five items are outstanding. They are listed in the order the specification asks for them to be worked — lowest priority number last — which is also, usefully, roughly the order of decreasing risk: the league work touches data and ordering, while the animation and navigation work is confined to the surface.

Item	State	Where the work is
10, 11, 12 — the Scholar League	Built, unproven	Engine, API and screen all written. The two required test scenarios are not written yet.
1 — premium animations	Not started	<code>index.html</code> , <code>css/styles.css</code> , <code>js/app.js</code>
2 — Update & News navigation	Not started	<code>css/styles.css</code> , and a check on a real device
14 — responsive pass	Not started	<code>css/styles.css</code> , on hardware
15, 16 — full debug and final verification	Ongoing	<code>test/</code> , plus a manual pass in a browser and on a phone

3.1 Priorities 10, 11 and 12 — the Scholar League

These three priorities share one engine, and the engine, the API and the screen are now all written. What is missing is proof: the two scenarios the specification names in Priority 15.6 have not been written as tests, so the fundamental requirement is currently satisfied by design and by reading, not by demonstration.

Already in place

- **The ranking rule (Priority 10).** `core.rankLeague()` in `js/goc-core.js` sorts by overall performance descending, then by XP descending, then by name. Performance therefore always outranks XP: a student with less XP but better results is placed above a student with more XP and poor results.
- **The performance calculation (Priority 11).** `core.computePerformance()` takes a student's sittings and returns one figure from objective, theory and JAMB-oriented results together, normalising over only the sections that student has actually been marked in. The weighting is configurable through `settings.perfWeights`, which defaults to 40 objective / 30 theory / 30 JAMB-oriented, rather than being hard-coded. The existing calculation was extended; there is no competing second one.
- **The data path (Priority 12).** `GOC.api.league()` returns rows, size, the signed-in student's own entry, a league name and the basis of the ranking, in both drivers, and `GET /api/league` serves the same thing. Deactivated accounts are filtered out of the table but keep their records, as Priority 7 requires. `core.leagueName()` maps a rank and roster size onto Diamond, Gold, Silver or Bronze.

- **The screen.** The `#league` section of `index.html` no longer carries typed-in podium names. It now has the hooks — `#lgPodium`, `#lgTable`, `#lgTier`, `#lgMineRank` and the rest — that `renderLeague()` in `js/app.js` fills from the API, reusing the existing `.lead-hero`, `.podium` and `.lrow` vocabulary with only a few `.lg-*` helpers added to the stylesheet. It re-reads on entry to the screen, at login, at sign-up, after a paper is submitted and after a console preview, so it cannot show a stale position.
- **The basis is stated on screen.** A line under the table explains that position is decided by academic performance and that XP only separates two scholars on the same performance — without it, a high-XP student sitting below a quieter classmate would read the board as broken.

Still to write

- **A test suite for the league (`test/test-p10.js`).** Build it on the same minidom and vm harness as `test/test-p9.js` and register it in `test/run.js`. It should assert that `renderLeague()` fills `#lgPodium` and `#lgTable` in exactly the order the API returned and never re-sorts, that the signed-in student's row carries the `.me` class and the "You" pill, that `#lgMineRank` reads the rank even when the student is far down the table, that a scholar with nothing marked reads "Not yet assessed" rather than 0.0%, and that an empty league shows the `.lg-empty` message instead of a blank card.
- **Scenario 1 of Priority 15.6, end to end.** A high-XP, low-performance student must appear **below** a low-XP, high-performance student. The way to build it without fixtures: sign in as Founder, unlock the console, and read the answer key from `listQuestions()`, which is staff-only and returns the correct answer. Then have one student sit several objective papers answering deliberately wrongly — each sitting still earns a flat share of XP, so eight or more papers gives a high XP total on poor results — while a second student sits one theory paper that the console marks full. The test must first assert the scenario is meaningful (that the first student really does hold more XP and less performance) before asserting the order, or it proves nothing.
- **Scenario 2 of Priority 15.6.** With performance equal, the higher XP must rank higher. Give a third student both a full-marked theory paper and an objective paper answered perfectly from the key, so performance ties at the top and XP alone separates them.
- **Ordering cases in `test-rules.js`.** Feed `core.rankLeague()` small hand-made rosters directly — the pure function, no DOM — so the rule is pinned at the level it is written at, including the name tie-break when performance and XP are both equal.
- **A league block in `test-server.js`.** Repeat the same guarantee over HTTP, so the order the server serves is checked and not only the order the demo driver computes. `GET /api/league` already exists; the block belongs beside the Priority 9 one, where a staff token and a student token are already in scope.

The one thing not to do here. The league screen must never sort what the API handed it. A second sort in `js/app.js` is exactly how XP would quietly end up outranking academic results — the specification calls performance-over-XP a fundamental requirement, and a stray comparator on the render side would break it invisibly, with the engine still passing all of its own tests. There is a comment to this effect above `renderLeague()`; leave it there.

3.2 Priority 1 — the premium animations

Nothing has been started here, and it is the largest piece of surface work left. Five parts are asked for, and the fifth is a constraint on the other four. The anchors each part attaches to already exist in the markup, so this is addition rather than restructuring.

Part	What is wanted	Notes for whoever builds it
1.1 — connected ecosystem	A three-part animation implying connection, collaboration, interaction, continuous flow and shared progress — one unified ecosystem. Animated connecting paths, flowing particles, progressive activation or coordinated transitions.	The section is already there and already says the right thing: <code>#lpFeatures</code> on the landing page, headed "One connected ecosystem — three parts working as one", with three <code>.lp-feat</code> cards for AI Literacy, Tutoring and the Web App. Add inline SVG paths between the three cards and put the motion in <code>css/styles.css</code> : no dependency, no canvas loop, and it degrades to a still diagram if the animation never runs.

Part	What is wanted	Notes for whoever builds it
1.2 — login transition	Login → Authentication → Successful Entry → Student Dashboard, triggered only after authentication succeeds.	login() in js/app.js already awaits the API and already shows the error state on failure. Wrap it, do not rewrite it: disable #loginBtn on submit to prevent a double send, run the sequence in the resolved branch only, and keep it short. No password or session token may appear in any label, toast or console line.
1.3 — sign-up transition	Account Creation → Account Setup → Successful Registration → Academy Hub.	signupSubmit() must keep its current order: validation, then the subject-combination rule, then account creation, then the Scholar ID, then the redirect. The animation goes around that, never between two steps of it. The generated Scholar ID is the natural thing to reveal at the end.
1.4 — the Academy Hub name	An animated text reveal, progressive appearance or light sweep on the Academy Hub name, reflecting learning, growth and continuous scholarship.	The wordmark is already a separate element from the mark: inside .lp-brand the logo is an <svg> using #goc-mark and the name is the .bt span beside it. Animate .bt (or a dedicated wordmark element in the hero) and nothing else. Do not put the animation on .lp-brand itself, because that container also holds the logo.
1.5 — do no harm	No layout breaking, no content displacement, no horizontal overflow, no broken navigation, no slow loading, no unresponsive controls, no mobile problems, no existing component failures. Responsive, and honouring reduced motion.	Animate transform and opacity only, never width, height, top or left. Give every animated block a stable reserved size so nothing reflows. The stylesheet already gates one effect behind @media (prefers-reduced-motion: no-preference); follow that existing pattern so the still state is the default and the motion is the enhancement — then test with reduced motion switched on.

The logo rule is absolute. The specification forbids modifying, animating, distorting, replacing, resizing or interfering with the existing Academy Hub logo, and names it again in the final verification step. Priority 1.4 animates the *name*; the mark itself stays exactly as it is, in its own element. Before reporting this priority done, confirm the icon files and the #goc-mark symbol and the markup around them are untouched.

3.3 Priority 2 — the Update & News navigation

The reported fault is that the All and New sections can block navigation to the other sections of the page. That fault has not yet been reproduced on a real device, so this is the one outstanding item that needs looking at before it needs coding. What the markup and stylesheet show is enough to narrow it to two candidates.

What the code shows

- The filter row is #upFilters in index.html, a .up-filters flex row of five .up-chip buttons — All, Admission, UTME, Scholarship, News — each calling filterUpdates(f, this).
- .up-filters in css/styles.css is display:flex with overflow-x:auto and no wrapping. On a narrow phone the five chips do not fit, so Scholarship and News sit off-screen to the right with no visible cue that they are there. A user who cannot see them reasonably reports that All and News are blocking the rest.
- The #updates screen also carries a sticky .lp-nav at z-index 40. If the filter row scrolls underneath it, the first chips stay reachable while the later ones are covered — the same symptom from a different cause.
- The landing-page drawer swipe handler in js/app.js was investigated and ruled out: it returns early unless the landing screen is the active one and the viewport is under 900px, so it cannot be swallowing taps on the updates screen.

The fix

- Open the page on an Android phone and a narrow desktop window and confirm which of the two candidates is real before changing anything.
- Let the row wrap below a breakpoint (flex-wrap:wrap, or a small auto-fit grid) so all five categories are visible at once and every one is tappable. Every existing section and all of its content stays — nothing may be hidden or removed to make room.
- If the row is kept as a scroller on the narrowest screens, that is permitted — horizontal scrolling is allowed for a component that is deliberately a scrolling tab strip — but it then needs a visible edge fade or a partly shown next chip so it is discoverable.
- Give the active chip an obvious state, stronger than the current .up-chip.on, so the reader always knows which category they are in.
- Check the stacking: the filter row must not be overlapped by the sticky nav, and the nav must not be overlapped by the row.
- Change nothing else on the page. The specification explicitly forbids redesigning unrelated parts of it.

3.4 Priority 14 — the responsive pass

Everything built in this round was written mobile-first and reuses the existing responsive vocabulary, but none of it has been opened on a physical phone. That pass is still owed, and the specification names the surfaces to check: the Web Test interface, question navigation, the timer, subject selection, the Update & News navigation, the League, the session notification, the animations and the dashboard navigation, across Android phones, small screens, tablets and desktop. The rule to hold to is that no horizontal scrolling may be introduced unless it is intentional, as in a scrolling tab strip. Four places are most likely to overflow and are worth checking first: the CBT question navigation panel, the console result tables, the four-figure row on the League's own standing card (which already falls to two columns under 360px), and the session warning bar.

3.5 Priorities 15 and 16 — debugging and final verification

Five of the specification's six test groups are covered by the automated suites. The sixth is the league, and the rest of Priority 16 cannot be finished until the features in 3.2 and 3.3 exist.

Test group	State
15.1 — authentication	Covered by test-rules.js, test-p78.js, test-p9.js and test-server.js.
15.2 — the Web Test system	Covered by test-p3.js, test-p4.js and test-server.js.
15.3 — subject personalisation	Covered by test-p6.js.
15.4 — account management	Covered by test-p78.js and test-server.js.
15.5 — session timeout	Covered. test-p9.js drives the warning, the countdown, the automatic logout, the guard on protected screens, a dropped connection and an unsubmitted paper; test-server.js proves the same over HTTP.
15.6 — the league	Not covered. Both required scenarios still need writing, as set out in 3.1. This is the single largest gap in the verification story.
16 — final verification	Outstanding. Needs a full suite run, the static checks, a manual pass in a browser and on a phone, and a confirmation that the logo is untouched.

Priority 15 says plainly not to assume the application works because the code compiles, and Priority 16 asks for a manual pass through every major flow. The automated suites drive the real interface rather than the internals, which is the right shape, but they run against a hand-built DOM stand-in and cannot see layout, paint or touch

behaviour. The manual pass is therefore not optional, and it is the last thing to do rather than the first. The flows worth walking are: sign up with a legal subject combination and with each illegal one; log in as a student, sit an objective paper, a theory paper and a JAMB-oriented CBT; mark the theory paper from the console and watch the score and the league move; deactivate and reopen an account; change the session policy as Founder and confirm a student is warned and signed out; and confirm the Founder-only controls refuse the Academic Director.

4. How to run and verify the project

Running it

- **Demo mode.** Open `index.html` directly. `js/api.js` sees a non-http protocol, short-circuits to the demo driver, and the whole application works in the page with the seeded roster. Nothing to install.
- **Server mode.** Run `node server/server.js` and open the address it prints, or use `start-server.ps1` on Windows. The boot probe finds `api/health` and switches to the HTTP driver. Data lives in `server/data.json`, which is created on first run.
- **Demo credentials.** Students `GOC-S-001 / amara2027!`, `GOC-S-002 / bello#88`, `GOC-S-003 / nwosu2026`. Management `GOC-A-001 / founder2027` (Founder) and `GOC-A-002 / acaddir2027` (Academic Director), with 2027 as the console access passcode. These are prototype fixtures and must be replaced before any real student uses the system.

Verifying it

- **The suites:** `node test/run.js` runs all seven and prints a per-suite and total count. It is currently 881 assertions with no failures — 101 in `test-rules.js`, 73 in `test-p6.js`, 169 in `test-p3.js`, 167 in `test-p4.js`, 90 in `test-p78.js`, 109 in `test-p9.js` and 172 in `test-server.js`. Run it before reporting any work as done.
- **Syntax:** `node --check` on `js/app.js`, `js/api.js`, `js/goc-core.js`, `js/pwa.js`, `sw.js` and `server/server.js`. `js/app.js` must stay ES5 — `var` and `function`, no arrow functions and no template literals — because the browser code is not transpiled.
- **The id scan:** a short script that checks every `getElementById` in `js/app.js` has a matching id in `index.html`. A handful of admin ids are injected by the console's own templates rather than written in the markup; those are expected and are the only ones that should be reported.

Why the tests look unusual. npm installs are refused in this environment, so `jsdom` is unavailable and `test/minidom.js` is a hand-written DOM stand-in. It has no events and no `removeChild` or `insertBefore`, so render code must rebuild through `innerHTML = "` and `appendChild`, which the application already does, and a form is driven by assigning `.value` and then calling the handler. Each suite runs the real js files inside one vm context whose sandbox is the window, and timers are stubbed, so anything time-based is stepped by hand. One bug in the stand-in itself was fixed this round: its parser was discarding text at the top level of a fragment, which silently truncated any markup of the form "`<b>Heading</b>` and the rest". If you later work somewhere npm is available, `jsdom` would be a fair replacement — but the suites are written to need nothing, and that is worth keeping.

5. The rules this work must not break

These are the specification's own prohibitions, restated as a checklist because every one is easy to break by accident while building what is left.

Rule	What it means in this codebase
The logo is untouched	Not modified, animated, distorted, replaced or resized. Priority 1.4 animates the Academy Hub name, which must stay a separate element from the mark.
Nothing existing is removed or redesigned	Extend, never rebuild. Every screen, feature, route and piece of branding that was there before must still be there and still work.
No duplicate systems	The Web Test question bank extended the existing question system rather than adding a second one, and the league extends <code>computePerformance</code> rather than introducing a rival calculation.

Rule	What it means in this codebase
No new dependencies	No package.json, no bundler, no CDN link. index.html must keep working when opened straight from disk.
Authorisation is server-side	Every rule is decided by the data layer before a handler runs, in both drivers. Hiding a control in the page is a courtesy, never the control.
Students cannot change scores, answers or ranking	Enforced and tested, including against forged scores and a forged clock. Any new route needs the same gate.
Passwords are hashed and never returned	No password, hash or passcode may be rendered, logged, echoed in a toast or returned by an API. The console shows the passcode as dots only.
Theory is never auto-scored	A theory paper carries no score until every written answer has been marked by hand. This was a real bug once; do not reintroduce it.
Performance outranks XP	In the league, always. XP breaks ties and nothing more — and the render side must not sort.
The session is enforced on the server	The page displays a countdown; it never decides access. Do not add a control that claims to extend a session the server will not extend.
History is preserved	Deactivating an account withdraws access only. Scholar IDs stay attached to every result, mark, performance figure and XP record.
UTME only	No other examination may be registered for, and a student who tries is told why on screen. The rule lives once in js/goc-core.js.
No in-app coins	Earning is XP, streaks and levels. Do not let a coin reappear in new copy or a new panel.

6. Decisions that are still yours

- **Fixed or sliding sessions.** As built, a session is a fixed length in both modes: neither driver extends it on activity, so a student reading quietly for two hours will still be signed out. That is the safer reading of “session time threshold” and the one to keep unless you say otherwise. If you want a sliding session instead, both drivers must change together and the warning should then appear only after genuine idleness — and in no case should there be a “Stay signed in” button that cannot actually extend anything.
- **The session numbers.** The default is a 120-minute student session with a 5-minute warning, clamped to 15–480 minutes and to a warning of 1–30 minutes that can never exceed the session. Management can change both from the console. Whether 120 and 5 are the right defaults for a cohort sitting long practice papers is a judgement to make, not code to write.
- **The performance weighting.** It currently sits at 40 objective, 30 theory and 30 JAMB-oriented. It is configurable rather than hard-coded, so it is a setting to agree, not code to change. Whether it should also be editable from the console is an open question.
- **The seeded demo data.** The roster, the demo passwords printed on the login screen and the sample questions are prototype fixtures. Before the pilot with real students, decide which of them should survive and which should be cleared — and the demo credentials on the login hint must go.
- **Two things outside this specification.** An image or mock-up of the web app was asked for earlier and has not been produced, and the five-subject structure (Use of English plus the four sciences) has not been confirmed as final. Neither has been started, because neither is part of the current brief.

Everything in the folder alongside this document is the working project, including the tests. Picking the work back up means opening section 3 at the first item, and running `node test/run.js` to confirm the 881 assertions still pass before touching anything.